



COSC/MATH 4340: Numerical Methods for Differential Equations

Ordinary Differential Equations (ODEs): Initial Value Problems (IVPs)

Class: COSC/MATH 4340

Semester: Spring 2024

Instructor: Dane Taylor

Office: Eng 4087

Email: dane.taylor@uwyo.edu





Introduction

Researchers and engineers often seek to solve an ordinary differential equation (ODE)

$$\frac{dy}{dt} = f(t, y).$$

Applications arise in all fields and include simulating a fluid flow, modeling disease spreading, solving robotics movements, or designing AI algorithms.

Often, the equation is too large or difficult to solve by hand, and we instead rely on approximating the solution using a computer.

Introduction



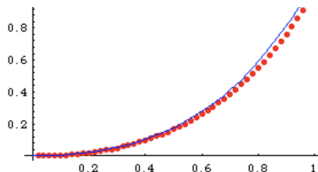
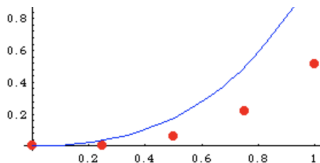
“Human computers” eventually replaced by electronic computers to predict, e.g., orbital motions.



Introduction

We can **numerically solve** $\frac{dy}{dt} = f(t, y)$ by obtaining a sequence of approximations $y_i \approx y(t_i)$ at times $t_i = t_0 + hi$, where $i = 0, 1, \dots, N$ and $h = (T - t_0)/N$ is the time step.

Smaller time steps h yield more accurate approximations but are more time consuming.





Introduction

Definition

An **initial value problem (IVP)** aims to solve an ODE with a given initial condition

$$\begin{cases} \frac{dy}{dt} = f(t, y) \\ y(t_0) = y_0 \end{cases}$$

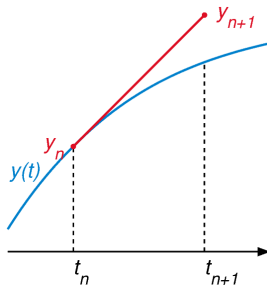
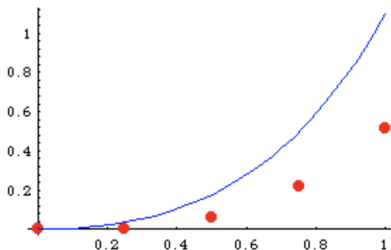
for a solution $y(t)$ over some range $t \in [t_0, T]$.

Definition

If $f(t, y) = f(t)$, i.e., it doesn't depend on y , then the ODE is called **autonomous**. Otherwise, it is **non-autonomous**

Forward Euler Method

The simplest numerical method to solve an IVP is **forward Euler**, which uses the slope $f(t_i, y_i)$ at y_i to predict y_{i+1} for each i



(See notes for derivation.)



Truncated Taylor Expansions

Consider the approximation of $y(t)$ near t_0

$$y(t) = y(t_0) + (t - t_0)y'(t_0) + \frac{(t - t_0)^2}{2}y''(t_0) + \frac{(t - t_0)^3}{3!}y'''(t_0) + \\ + \frac{(t - t_0)^4}{4!}y^{(4)}(t_0) + \dots$$

The intermediate value theorem implies that there exists a value $\xi \in [t_0, t]$ allowing truncation without error:

$$y(t) = y(t_0) + (t - t_0)y'(t_0) + \frac{(t - t_0)^2}{2}y''(\xi)$$

$$y(t) = y(t_0) + (t - t_0)y'(t_0) + \frac{(t - t_0)^2}{2}y''(t_0) + \frac{(t - t_0)^3}{2}y'''(\xi).$$

(See notes and code.)



Forward Euler

Definition

The **forward Euler method** approximately solves an IVP by $y_0 = y(t_0)$ and

$$y_{i+1} = y_i + hf(t_i, y_i), \quad i = 0, 1, 2, \dots, N$$

with $h = (T - t_0)/N$ and $t_i = t_0 + hi$.



Forward Euler

Example: Numerically solve

$$\begin{cases} \frac{dy}{dt} = f(t, y) = 2y \\ y(t_0) = y_0 = 1 \end{cases}$$

over the range $t \in [0, 1]$ using the FE method with $N = 5$ steps.

(See notes and code.)



Existence, Uniqueness, and Stability

Before attempting to solve or approximate $y(t)$, we should know if the solution $y(t)$ exists and is unique. This requires a strong notion for what “continuous” means

Definition

A function $f(t, y)$ is **Lipschitz continuous** over a region $\mathcal{S} = [t_0, T] \times [\alpha, \beta]$ if there exists a **Lipschitz constant** $L > 0$ such that

$$|f(t, y_1) - f(t, y_2)| \leq L|y_1 - y_2|$$

for any $t \in [t_0, T]$ and $y_1, y_2 \in [\alpha, \beta]$.

Note that f varies continuously with variable y in a very specific (and boundable) way



Existence, Uniqueness, and Stability

Lipschitz continuity example with $f(t, y) = ty + t^3$.

(See notes.)



Existence, Uniqueness, and Stability

Lipschitz continuity allows for an existence and uniqueness result

Theorem

Assume $f(t, y)$ is Lipschitz continuous over a region $\mathcal{S} = [t_0, T] \times [\alpha, \beta]$. Then the IVP with initial condition $y(t_0) = y_0$ for some $y_0 \in [\alpha, \beta]$ has exactly one solution $y(t)$ over the range $t \in [t_0, c]$ for some $c \in [t_0, T]$. Moreover, if f is Lipschitz continuous over the region $\mathcal{S} = [t_0, T] \times [-\infty, \infty]$, then the IVP has exactly one solution over $t \in [t_0, T]$.



Existence, Uniqueness, and Stability

Lipschitz continuity allows for a stability result

Theorem

Assume $f(t, y)$ is Lipschitz continuous over a region $\mathcal{S} = [t_0, T] \times [\alpha, \beta]$ with Lipschitz constant L , and consider two solutions $Y(t)$ and $Z(t)$ that solve the ODE with initial conditions $Y(t_0) = Y_0$ and $Z(t_0) = Z_0$, then

$$|Y(t) - Z(t)| \leq e^{L(t-t_0)} |Y_0 - Z_0|$$

If $Y(t)$ and $Z(t)$ are two **trajectories** with two different initial conditions, then the difference between the trajectories exponentially grows with t and rate L

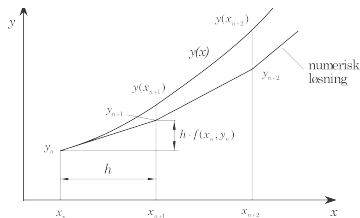
Local and Global Error

Definition

The **local truncation error** e_i for an IVP solver is the error the occurs at 1 step (assuming the previous step has zero error).

Definition

The **global error** E_i for an IVP solver is the error that accumulates across the iterated steps.





Local and Global Error

Recall the **forward Euler method** $y_{i+1} = y_i + hf(t_i, y_i)$

Theorem

The **local truncation error** e_i for the forward Euler method is second order, $e_i = \mathcal{O}(h^2)$, and if $|y''(t)| \leq M$ for some constant M , then $e_i \leq \frac{Mh^2}{2}$ for all i

Theorem

Considering an IVP with a Lipschitz function with Lipschitz constant L , the **global error** E_i for the forward Euler method is first order, $E_i = \mathcal{O}(h)$, and if $|y''(t)| \leq M$ for some constant M , then

$$E_i \leq \frac{Mh}{2L}(e^{Lhi} - 1)$$

(See notes for proof.)



Local and Global Error

Study error for earlier example: $f(t, y) = 2y(t)$.

(See notes and code.)



Local and Global Error

Theorem [General Error Bound]

Consider an IVP solver for a Lipschitz function with Lipschitz constant L in which the local truncation error satisfies the bound $e_i \leq ch^{k+1}$ for some constants c and k . Then the **global errors** E_i satisfy the bound

$$E_i \leq \frac{ch^k}{L}(e^{Lhi} - 1)$$

The proof is almost identical to the analysis of global error for forward Euler (which is recovered under the choice $k = 1$ and $c = M/2$).



Higher-Order Taylor Methods

Forward Euler is a 1st-order method: local error is $\mathcal{O}(h^2)$ and global error is $\mathcal{O}(h)$.

Taylor methods are one family of techniques to create higher-order accuracy. They rely on higher-order Taylor expansions:

$$y(t) = y(t_0) + (t - t_0)y'(t_0) + \frac{(t - t_0)^2}{2}y''(t_0) + \frac{(t - t_0)^3}{3!}y'''(t_0) + \frac{(t - t_0)^4}{4!}y^{(4)}(t_0) + \dots$$



Taylor Methods

Definition

A k th-order **Taylor method** solves an IVP with the iteration

$$y_{i+1} = y_i + hf(t_i, y_i) + \frac{h^2}{2}f'(t_i, y_i) + \dots + \frac{h^k}{k!}f^{(k-1)}(t_i, y_i)$$

The “prime” notation $f'(t, y)$ is the **total derivative**, which can be obtained with **partial derivatives** $f_t(t, y)$ and $f_y(t, y)$:

$$f'(t, y) = \frac{d}{dt}f(t, y(t)) = f_t + f_y \frac{\partial y}{\partial t} = f_t + f_y f$$
$$f'(t_i, y_i) = f_t(t_i, y_i) + f_y(t_i, y_i)f(t_i, y_i)$$

(See note for $f''(t, y)$)



Local and Global Error

Theorem

A k th-order Taylor method has **local truncation error** $e_i \leq ch^{k+1}$ for all i , where $c = \frac{M}{(k+1)!}$ and $|y^{(k+1)}(t)| \leq M$.

Theorem

The **global error** E_i for a k -th order Taylor method is bound by

$$E_i \leq \frac{ch^{(k)}}{L}(e^{Lhi} - 1)$$

(See notes for proof.)

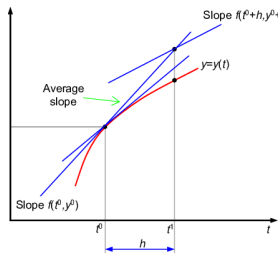
Explicit Trapezoidal Method

Definition

The **explicit trapezoidal method** solves an IVP with the iteration

$$\tilde{y}_{i+1} = y_i + hf(t_i, y_i)$$

$$y_{i+1} = y_i + \frac{h}{2} [f(t_i, y_i) + f(t_{i+1}, \tilde{y}_{i+1})]$$





Local and Global Error

Theorem

The **local truncation error** e_i for the explicit trapezoidal method is third order, $e_i = \mathcal{O}(h^3)$. and if $|y'''(t)| \leq M$ for some constant M , then $e_i \leq ch^3$ for all i for some c

Theorem

Considering an IVP with a Lipschitz function with Lipschitz constant L , the **global error** E_i for the explicit trapezoidal method is bound by

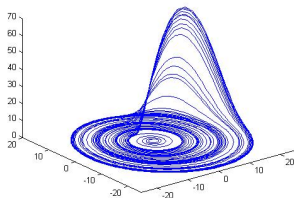
$$E_i \leq \frac{ch^2}{L}(e^{Lhi} - 1)$$

(See notes for proof.)

Solving a System of ODEs

How do we solve a system, such as a Rossler system with 3 variables $x(t)$, $y(t)$ and $z(t)$ and 3 coupled differential equations?

$$\begin{cases} \frac{dx}{dt} = f_1(x, y, z) = -y - z \\ \frac{dy}{dt} = f_2(x, y, z) = x + ay \\ \frac{dz}{dt} = f_3(x, y, z) = b + z(x - c). \end{cases}$$





Solving a System of ODEs

Definition

A **system of ODEs** is a set of ODEs describing coupled functions $y_1(t), \dots, y_p(t)$. Letting $\mathbf{y}(t) = [y_1(t), \dots, y_p(t)]$, each equation takes the form

$$\frac{dy_j}{dt} = f_j(t, \mathbf{y}(t)), \quad j = 1, 2, \dots, p$$

and the entire system can be written as

$$\frac{d\mathbf{y}(t)}{dt} = F(t, \mathbf{y}(t)),$$

where $F(t, \mathbf{y}(t)) = [f_1(t, \mathbf{y}(t)), \dots, f_p(t, \mathbf{y}(t))]$.

Forward Euler for a System of ODEs

Definition

An **initial value problem (IVP)** for a **system of ODEs** aims to solve for the functions $y_j(t)$ over a domain $t \in [t_0, T]$, and it requires a set of initial conditions $y_1(t_0), \dots, y_n(t_0)$.

Definition

The **forward Euler method for a system of ODEs** approximates the solutions $y_j(t)$ at times $t_i = t_0 + hi$ with $h = (T - t_0)/N$ by setting $y_{0,j} = y_j(t_0)$ and iterating

$$y_{i+1,j} = y_{i,j} + hf_j(t_i, \mathbf{y}_i), \quad i = 0, 1, 2, \dots, N, \quad j = 1, 2, \dots, p$$

where $\mathbf{y}_i = [y_{i,1}, \dots, y_{i,p}]$.



Forward Euler for a System of ODEs

Example: Solve the susceptible-infected-susceptible (SIS) epidemic model. Let $S(t)$ be the fraction of susceptible persons and $I(t)$ be the fraction of infected persons. Let $\mathbf{y}(t) = [S(t), I(t)]$ and define

$$\begin{aligned}\frac{dS(t)}{dt} &= f_1(t, [S(t), I(t)]) = -\beta I(t)S(t) + \gamma I(t) \\ \frac{dI(t)}{dt} &= f_2(t, [S(t), I(t)]) = \beta I(t)S(t) - \gamma I(t)\end{aligned}$$

where $\beta > 0$ is the transmission rate and $\gamma > 0$ is the healing rate.

(See notes and code.)



Existence and Uniqueness of a Solution

Lipschitz continuity also applies to vector-valued functions

Definition

A vector-valued function $F(t, \mathbf{y})$ is **Lipschitz continuous** over a region $\mathcal{S} = [t_0, T] \times \mathcal{D}$ if there exists a **Lipschitz constant** $L > 0$ such that

$$\|F(t, \mathbf{y}) - F(t, \mathbf{z})\| \leq L\|\mathbf{y} - \mathbf{z}\|$$

for any $t \in [t_0, T]$ and $\mathbf{y}, \mathbf{z} \in \mathcal{D}$.

Theorem

Assume $F(t, \mathbf{y})$ is Lipschitz continuous over a region $\mathcal{S} = [t_0, T] \times \mathbb{R}^p$. Then the IVP with initial condition $\mathbf{y}(t_0) = \mathbf{y}^{(0)}$ has exactly one solution $\mathbf{y}(t)$ over $t \in [t_0, T]$.



Forward Euler for Higher-order ODEs

We have so far focused on solving 1st order differential equations:

$$y'(t) = \frac{dy}{dt}(t) = f(t, y(t))$$

We can also solve a k -th order ODE

$$y^{(k)}(t) = \frac{d^k y}{dt^k}(t) = f(t, y(t), y'(t), y''(t), \dots, y^{(k)}(t))$$

by transforming it into a system of ODEs

$$y_1(t) = y(t)$$

$$y_2(t) = y'(t) = \frac{dy}{dt}(t)$$

$$y_3(t) = y''(t) = \frac{d^2 y}{dt^2}(t)$$

$$\vdots$$

$$y^{(k)}(t) = \frac{d^k y}{dt^k}(t)$$



Forward Euler for Higher-order ODEs

Example: Solve

$$y'''(t) = \frac{d^3y}{dt^3}(t) = y(t) + 2y'(t) + y''(t)$$

with initial conditions $y(0) = 1$, $y'(0) = 1$ and $y''(0) = 0$

***Note this is a linear, third-order differential equation.

(See notes and code.)



Runge-Kutta (RK) Methods

Recall that the Trapezoidal method has a smaller error than forward Euler, but note it requires finding the slope at two different times: t_i and t_{i+1} (i.e., 2 “steps”).

Definition

Runge-Kutta (RK) methods are explicit “multistage” methods for solving an IVP that take the form

$$y_{i+1} = y_i + h\tilde{f}(t_i, y_i),$$

where $\tilde{f}(t_i, y_i)$ is “best” slope that is obtained as a weighted average of k different slopes (i.e., k “stages”).

For the trapezoidal method, $\tilde{f}(t_i, y_i) = \frac{1}{2} [f(t_i, y_i) + f(t_{i+1}, \tilde{y}_{i+1})]$



Midpoint Methods

Definition

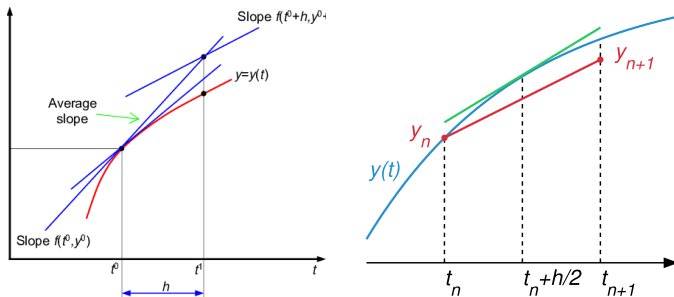
The **midpoint method** is a 2-stage method that solves an IVP with the iteration

$$\begin{aligned}\tilde{y}_{i+1/2} &= y_i + \frac{h}{2}f(t_i, y_i) \\ y_{i+1} &= y_i + hf(t_i + h/2, \tilde{y}_{i+1/2})\end{aligned}$$



2nd Order Runge-Kutta Method (RK2)

The Trapezoidal and Midpoint methods are Runge-Kutta methods with 2 stages



(See note.)



Local and Global Error

Midpoint's error is similar to trapezoidal's:

Theorem

The **local truncation error** e_i for the midpoint method is third order, $e_i = \mathcal{O}(h^3)$. and if $|y'''(t)| \leq M$ for some constant M , then $e_i \leq Ch^3$ for all i for some C

Theorem

Considering an IVP with a Lipschitz function with constant L , the **global error** E_i for the midpoint method is bound by

$$E_i \leq \frac{Ch^2}{L}(e^{Lhi} - 1)$$

(See notes for proof.)



4th Order Runge-Kutta Method (RK4)

Definition

The **4th-order Runge-Kutta method (RK4)** solves an IVP with $y_0 = y(t_0)$ and the iteration

$$y_{i+1} = y_i + \frac{h}{6} [k_1 + 2k_2 + 2k_3 + k_4]$$

$$k_1 = f(t_i, y_i)$$

$$k_2 = f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2}k_1\right)$$

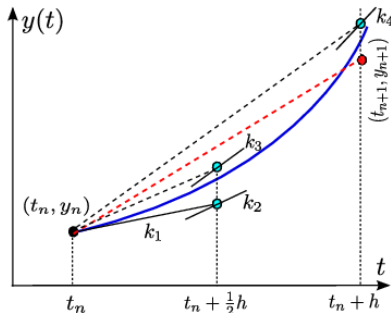
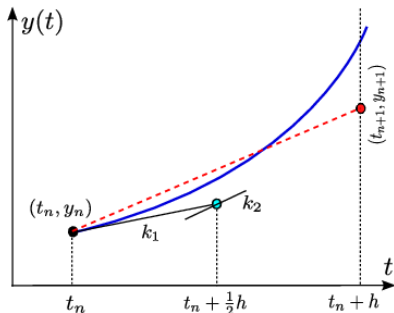
$$k_3 = f\left(t_i + \frac{h}{2}, y_i + \frac{h}{2}k_2\right)$$

$$k_4 = f(t_i + h, y_i + hk_3)$$



4th Order Runge-Kutta Method (RK4)

Comparison of RK2 and RK4





4th Order Runge-Kutta Method (RK4)

Example: Solve

$$y'(t) = (t - y)/2$$

over $t \in [0, 10]$ with initial condition $y(0) = 1$.

(See notes and code.)



Error for 4th Order Runge-Kutta (RK4)

Theorem

The **local truncation error** e_i for RK4 is fifth order, $e_i = \mathcal{O}(h^5)$ so that $e_i \leq ch^5$ for all i for some c

Theorem

Given an IVP with a function $f(t, y)$ with Lipschitz constant L , the **global error** E_i for the midpoint method is bound by

$$E_i \leq \frac{ch^4}{L}(e^{Lhi} - 1)$$

(See notes for context.)



RK Pairs and Adaptive Step Sizes

Thus far, we studied approximation error $E_i = |y(t_i) - y_i|$ using the true function $y(t)$, which is typically unknown.

In practice, we must approximate E_i by comparing one approximation y_i to a more accurate approximation, such as one obtained using smaller time steps or a higher-order method.

Definition

RK Pairs allow one to efficiently and simultaneously develop both a higher-accuracy and a lower-accuracy predict together.

RK Pairs and Adaptive Step Sizes

Definition

The **RK 2/3 Method** simultaneously provides both 2nd-order and 3rd-order approximations to an IVP with $y_0 = y(t_0)$ and using the iteration

$$y_{i+1} = y_i + h \frac{(s_1 + s_2)}{2} \quad (\text{trapezoidal})$$

$$z_{i+1} = z_i + \frac{h}{6} [s_1 + s_2 + 4s_3]$$

$$s_1 = f(t_i, y_i)$$

$$s_2 = f(t_i + h, y_i + hs_1)$$

$$s_3 = f\left(t_i + 0.5h, y_i + 0.5h \frac{(s_1 + s_2)}{2}\right)$$



RK Pairs and Adaptive Step Sizes

One can estimate the error $E_{i+1} = |y_{i+1} - z_{i+1}|$ and adjust the time step size h to ensure the error is smaller than some tolerance ϵ .

For example, if $E_{i+1} > \epsilon$ is too large, one can decrease h by a factor qh with $q < 1$ to decrease the new error $\hat{E}_i$ below ϵ

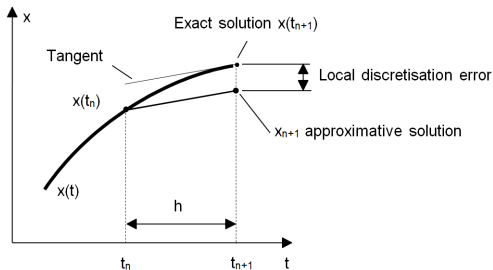
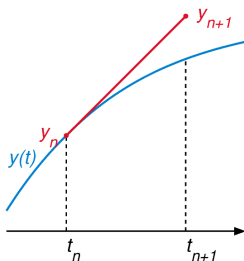
If we know the method is order k , i.e., $E_i \leq ch^k$ for some constant c , then $\hat{E}_i$ for step size qh becomes $\hat{E}_i \leq c(qh)^k = q^k E_i < \epsilon$ if

$$q < (\epsilon/E_i)^{1/k}.$$



Backward Euler Method

The backward Euler method is similar to forward Euler, but one uses the slope at the future time step t_{i+1} rather than time t_i . Below: forward vs backward Euler.





Backward Euler Method

Definition

The **backward Euler method** approximately solves an IVP by $y_0 = y(t_0)$ and

$$y_{i+1} = y_i + hf(t_{i+1}, y_{i+1}), \quad i = 0, 1, 2, \dots, N$$

with $h = (T - t_0)/N$ and $t_i = t_0 + hi$.

Note that the y_{i+1} is a function of y_{i+1} , and so it must be found with a root-finding algorithm.

Yes it is slower, but it has a benefit ... (good for a family of problems called “stiff” equations.)

(See notes for derivation.)



Fixed Point Iteration

There are many ways to find a value x such that $x = g(x)$. A common one is fixed point iteration.

Definition

Let r denote a fixed point of $g(x)$ so that $g(x^*) = x^*$. **Fixed point iteration** constructs a converging sequence to x^* given an initial guess x_0 via the iteration

$$x(t+1) = g(x(t)).$$

Theorem

The iterations x_t converge to x^ iff $|\frac{dg}{dx}| < 1$ at $x = x^*$.*

(See code.)



Backward Euler Method

Example. Solve $f(t, y) = 2y$ and $y(0) = 1$ with backward Euler.

(See notes and code.)



Backward Euler Errors

The error is similar to that of forward Euler

Theorem

The **local truncation error** e_i for the backward Euler method is second order, $e_i = \mathcal{O}(h^2)$, and if $|y''(t)| \leq M$ for some constant M , then $e_i \leq ch^2$ for some c

Theorem

Considering an IVP with a Lipschitz function with Lipschitz constant L , the **global error** E_i for the backward Euler method is first order, $E_i = \mathcal{O}(h)$, then

$$E_i \leq \frac{ch}{L}(e^{Lhi} - 1)$$

(See notes for proof.)



Backward Euler for Higher-order ODEs

Example: Revisit the example

$$y'''(t) = \frac{d^3y}{dt^3}(t) = y(t) + 2y'(t) + y''(t)$$

with initial conditions $y(0) = 1$, $y'(0) = 1$ and $y''(0) = 0$

(See notes.)



Implicit vs Explicit Methods

Definition

An IVP solver is called an **explicit method** if the next iteration only depends on past iterations

$$y_{i+1} = g(t, y_i, y_{i-1}, y_{i-2}, \dots)$$

Definition

An IVP solver is called an **implicit method** if the next iteration is a function of itself as well as past iterations

$$\underline{y_{i+1}} = g(t, \underline{y_{i+1}}, y_i, y_{i-1}, y_{i-2}, \dots)$$

See examples.

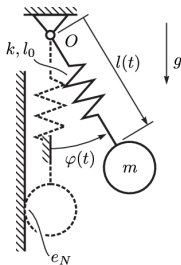


Implicit vs Explicit Methods

Definition

An IVP is called a **stiff** if the ODE has multiple timescales.

Implicit methods are better at solving “stiff” equations. Explicit methods can solve a stiff problem only if the time step h is very, very small.





General Multistep Methods

Definition

A numerical method to solve an IVP is called **multistep** if multiple past y_i estimates are used to predict the next y_{i+1} estimate.

Definition

The **Adams-Bashforth** methods are explicit multistep methods that take the form

$$y_{i+1} = \sum_{k=1}^K a_k y_{i+1-k} + h \sum_{k=1}^K b_k f(t_{i+1-k}, y_{i+1-k})$$



General Multistep Methods

Implicit multistep methods are very similar and take the form:

Definition

The **Adams-Moulton** methods are implicit multistep methods that take the form

$$y_{i+1} = \sum_{k=1}^K a_k y_{i+1-k} + h \sum_{k=0}^K b_k f(t_{i+1-k}, y_{i+1-k})$$

so that there is an implicit term $f(t_{i+1}, y_{i+1})$.

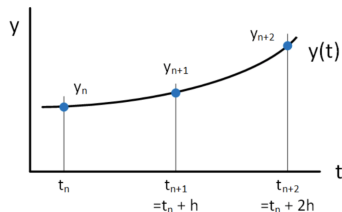
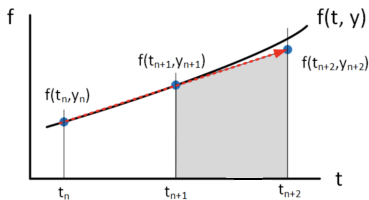
Their derivations are nearly identical to that for the Adams-Bashford methods.



General Multistep Methods

Example: 2nd order Adams-Bashforth method

$$\begin{aligned}y_{i+1} &= a_1 y_i + a_2 y_{i-2} + b_1 h f(t_i, y_i) + b_2 h f(t_{i-1}, y_{i-1}) \\&= y_i + 0 + \frac{3h}{2} f(t_i, y_i) - \frac{h}{2} f(t_{i-1}, y_{i-1}) \\&= y_i + \frac{h}{2} [3f(t_i, y_i) - f(t_{i-1}, y_{i-1})]\end{aligned}$$





General Multistep Methods

3rd order Adams-Bashforth method

$$y_{i+1} = y_i + \frac{h}{12}[23f(t_i, y_i) - 16f(t_{i-1}, y_{i-1}) + 5f(t_{i-2}, y_{i-2})]$$

4th order Adams-Bashforth method

$$y_{i+1} = y_i + \frac{h}{24}[55f(t_i, y_i) - 59f(t_{i-1}, y_{i-1}) + 37f(t_{i-2}, y_{i-2}) - 9f(t_{i-3}, y_{i-3})]$$

In fact there are many 2nd order Adams-Bashforth methods

$$y_{i+1} = \frac{1}{2}[y_i + y_{i-1}] + \frac{h}{4}[7f(t_i, y_i) - f(t_{i-1}, y_{i-1})]$$

(See notes and code.)



Stability of Multistep Methods

Definition

A multistep method is called **stable** if the roots of polynomial

$$Q(x) = x^K - a_1x^{K-1} - a_2x^{K-2} - \dots - a_{K-1}x - a_K$$

have magnitude ≤ 1 (and are simple if $= 1$).

Definition

If there is only root with magnitude 1 and it is 1, then the method is **strongly stable**.

Definition

If there are more roots with magnitude 1, then it's **weakly stable**.

(See notes and code.)



General Multistep Methods

Example: 2-step Adams-Moulton method

$$\begin{aligned}y_{i+1} &= a_1 y_i + a_2 y_{i-1} + b_0 h f(t_{i+1}, y_{i+1}) + b_1 h f(t_i, y_i) + b_2 h f(t_{i-1}, y_{i-1}) \\&= y_i + 0 + \frac{h}{12} [5f(t_{i+1}, y_{i+1}) + 8f(t_i, y_i) - f(t_{i-1}, y_{i-1})] \\&= y_i + \frac{h}{12} [5f(t_{i+1}, y_{i+1}) + 8f(t_i, y_i) - f(t_{i-1}, y_{i-1})]\end{aligned}$$

Claim: It's 3rd order and has bounded local error

$$e_i \leq \frac{Mh^4}{24}, \quad M = \max_t |y^{(4)}(t)|.$$

(See notes.)



Stability of Multistep Methods

Let's characterize how error vanishes as h decreases.

Definition

An IVP solver is **consistent** if:

$$\lim_{h \rightarrow 0} \max_i \left| \frac{e_i}{h} \right| = 0$$

Definition

An IVP solver is **convergent** if:

$$\lim_{h \rightarrow 0} \max_i |E_i| = 0$$

See notes.



Stability of Multistep Methods

Theorem [Dahlquist]

Assuming the initial values are correct, a multistep method is convergent iff it is consistent and stable.

Nearly all IVP solvers are consistent (i.e., at least as good as forward Euler). Thus, understanding a methods stability determines if it is convergent.



Stability of Multistep Methods

In fact there are several types important types of “stability.”

Definition

Absolute stability (or linear stability): The system is convergent under the choice $f(t, y) = \lambda y$.

This can be checked by examining roots of a polynomial $Q(x, h\lambda)$ associated with the IVP solver under the ansatz $y_i = cx^i$.

Theorem

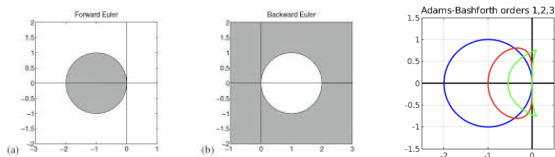
An IVP solver is **absolute stable** iff its characteristic polynomial $Q(x, h\lambda)$ is stable.

(See notes for examples.)

Stability of Multistep Methods

Definition

The **region of absolute stability** for an IVP solver is the domain of the complex parameter space $(\operatorname{Re}(h\lambda), \operatorname{Im}(h\lambda))$ in which the method is absolutely stable.



Above are images of “regions of stability” for three IVP solvers, i.e., the values of $h\lambda$ for which the method is stable (See notes.)



Stability of Multistep Methods

Definition

A method is **A stable** if its region of absolute stability contains the entire left-half plane. (These are implicit methods that can solve stiff equations with large time steps.)

